OAVS • TGT COMPUTER SCIENCE TEACHER EXAMINATION

MCQ Question Bank

Other Features & Miscellaneous Functions

TOTAL QUESTIONS

190 Unique MCQs | 13 Sections

`atof()` `atoi()` `toupper()` `tolower()`
`isalnum()` `isalpha()` `isdigit()` `exit()`

Strictly limited to the prescribed syllabus • With Answers & Explanations

Table of Contents

Overview of Miscellaneous Functions	8 Questions
atof() - ASCII to Float Conversion	18 Questions
atoi() - ASCII to Integer Conversion	18 Questions
toupper() - Convert to Uppercase	14 Questions
tolower() - Convert to Lowercase	14 Questions
isalnum() - Alphanumeric Test	14 Questions
isalpha() - Alphabetic Test	13 Questions
isdigit() - Decimal Digit Test	13 Questions
exit() - Program Termination	16 Questions
Comparison of Listed Functions	14 Questions
Code and Output-Based Questions	18 Questions
Difficult and Tricky Questions	14 Questions
Mixed OAVS TGT Practice	16 Questions

How to use this bank: Each question card shows the question, four options (A-D), the correct answer strip, and a short exam-oriented explanation for quick revision. Answer options are randomly distributed across A, B, C, and D throughout the bank.

Overview of Miscellaneous Functions

Q1 OVERVIEW

Which header file declares atoi(), atof(), and exit()?

A. stdlib.h

B. stdio.h

C. string.h

D. ctype.h

Ans: A

Explanation: atoi(), atof(), and exit() are all declared in the standard library header stdlib.h.

Q2 OVERVIEW

Which header file declares toupper(), tolower(), isalpha(), isalnum(), and isdigit()?

A. conio.h

B. ctype.h

C. stdio.h

D. stdlib.h

Ans: B

Explanation: Character classification and conversion functions belong to the ctype.h header.

Q3 OVERVIEW

Which of the following functions performs character testing rather than character conversion?

A. isalpha()

B. tolower()

C. toupper()

D. atoi()

Ans: A

Explanation: isalpha() tests a character's classification; it does not convert or change it.

Q4 OVERVIEW

Which of the following functions performs string-to-number conversion?

A. isdigit()

B. exit()

C. toupper()

D. atof()

Ans: D

Explanation: atof() converts a numeric string into a floating-point (double) value.

Q5 OVERVIEW

Which function among the following is used to terminate a program?

A. isalnum()

B. exit()

C. tolower()

D. atoi()

Ans: B

Explanation: exit() immediately terminates program execution and returns a status to the OS.

Q6 OVERVIEW

How many functions are covered under "Other Features and Miscellaneous Functions" in this syllabus?

A. Eight

B. Six

C. Ten

D. Seven

Ans: A

Explanation: The syllabus lists exactly eight functions: atof, atoi, toupper, tolower, isalnum, isalpha, isdigit, and exit.

Q7 OVERVIEW

Which pair of functions both accept a character (as int) and return an integer true/false result?

A. atoi() and atof()

B. atof() and atoi()

C. exit() and atoi()

D. isalpha() and isdigit()

Ans: D

Explanation: isalpha() and isdigit() are character-testing functions that return a nonzero or zero integer result.

Q8 OVERVIEW

Which of these functions does NOT return control/a value to the calling program?

A. atoi()

B. toupper()

C. isalpha()

D. exit()

Ans: D

Explanation: exit() terminates the entire program and never returns control to the point from which it was called.

atof() - ASCII to Float Conversion

Q9 ATOF()

What does the name atof() functionally represent?

A. Array to float

B. ASCII to float

C. Auto to float

D. ASCII to fixed

Ans: B

Explanation: atof() converts an ASCII string representation of a number into a floating-point (double) value.

Q10 ATOF()

What is the return type of atof()?

A. double

B. int

C. char*

D. float

Ans: A

Explanation: The C standard defines atof() to return a value of type double.

Q11 ATOF()

What type of argument does atof() accept?

A. single char

B. double*

C. int

D. const char* (a string)

Ans: D

Explanation: atof() takes a pointer to a null-terminated character string as its only argument.

Q12 ATOF()

Which header must be included to use atof()?

A. stdlib.h

B. ctype.h

C. math.h

D. stdio.h

Ans: A

Explanation: atof() is declared in stdlib.h, the standard general-utilities header.

Q13 ATOF()

What value does atof("12.5") return?

A. "12.5" (string)

B. 12

C. 0.0

D. 12.5

Ans: D

Explanation: atof() parses the numeric string "12.5" and returns the double value 12.5.

Q14 ATOF()

What value does atof() return if the string contains no valid numeric conversion at all?

A. NULL

B. 0.0

C. An undefined garbage value

D. -1.0

Ans: B

Explanation: If no valid conversion can be performed, atof() is defined to return 0.0.

Q15 ATOF()

What will atof("abc") return?

A. "abc" unchanged

B. -1

C. A runtime error

D. 0.0

Ans: D

Explanation: Since "abc" has no valid leading numeric portion, atof() returns 0.0.

Q16 ATOF()

What does atof(" 45.6") return, given a leading space?

A. 45.6

B. A compilation error

C. " 45.6" unchanged

D. 0.0

Ans: A

Explanation: atof() automatically skips leading whitespace characters before beginning conversion.

Q17 ATOF()

What is the result of atof("3.14xyz")?

A. 3.14xyz (unchanged)

B. A runtime error

C. 0.0

D. 3.14

Ans: D

Explanation: atof() converts only the initial valid numeric portion and silently ignores the trailing non-numeric characters.

Q18 ATOF()

Which of the following strings can atof() correctly convert to a non-zero value?

A. "-2.5e3"

B. "##2.5"

C. "$25.00"

D. "twenty-five"

Ans: A

Explanation: atof() supports an optional sign and scientific (exponential) notation, so "-2.5e3" converts to -2500.0.

Q19 ATOF()

How does atof() treat a leading '+' or '-' sign in the input string?

A. It correctly interprets the sign as part of the number

B. It ignores the sign completely

C. It treats the sign as a separate character

D. It treats the sign as an error

Ans: A

Explanation: atof() recognizes an optional leading sign immediately before the digits of the number.

Q20 ATOF()

Which of the following statements about atof() is true?

A. It requires the string to contain only digits

B. It returns a pointer to the unconverted remainder

C. It does not modify the original input string

D. It modifies the string in place during conversion

Ans: C

Explanation: atof() only reads the string to compute a double value; the original string argument is left unchanged.

Q21 ATOF()

A programmer stores the result of atof() into an int variable. What is the likely consequence?

A. The program fails to compile

B. atof() throws a runtime exception

C. The int variable automatically becomes a double

D. The fractional part of the converted value is lost (truncated)

Ans: D

Explanation: Assigning a double result to an int variable truncates any decimal portion of the value.

Q22 ATOF()

What is the output of: printf("%.1f", atof("7.8g9"));

A. A runtime error

B. 0.0

C. 7.8

D. 7.89

Ans: C

Explanation: Conversion stops at the first invalid character 'g', so only the leading "7.8" portion is converted.

Q23 ATOF()

Which function is best suited to convert the string "99.99" into a usable numeric value for a price calculation?

A. toupper()

B. atoi()

C. isdigit()

D. atof()

Ans: D

Explanation: atof() is designed specifically to convert decimal/floating-point numeric strings.

Q24 ATOF()

What does atof("") (an empty string) return?

A. 0.0

B. A runtime error

C. Undefined behaviour

D. NULL

Ans: A

Explanation: With no digits available to convert, atof() is defined to return 0.0 for an empty string.

Q25 ATOF()

Assertion (A): atof() can convert strings containing exponential notation like "1e5".

Reason (R): atof() follows the same numeric grammar as strtod(), which supports exponents.

A. A is true, R is false

B. A is false, R is true

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: C

Explanation: atof() is essentially strtod() without error reporting, so it also supports scientific/exponential notation.

Q26 ATOF()

Fill in the blank: atof() converts the initial portion of a string to a ___ value.

A. integer

B. string

C. double (floating-point)

D. character

Ans: C

Explanation: By definition, atof() converts a numeric string prefix into a double (floating-point) value.

atoi() - ASCII to Integer Conversion

Q27 ATOI()

What does the name atoi() functionally represent?

A. ASCII to integer

B. ASCII to index

C. ASCII to iterate

D. Array to integer

Ans: A

Explanation: atoi() converts an ASCII numeric string into an equivalent integer value.

Q28 ATOI()

What is the return type of atoi()?

A. char

B. long

C. int

D. double

Ans: C

Explanation: atoi() is defined to return a value of type int.

Q29 ATOI()

Which header file declares atoi()?

A. stdlib.h

B. string.h

C. ctype.h

D. stdio.h

Ans: A

Explanation: atoi() is declared, along with atof() and exit(), in stdlib.h.

Q30 ATOI()

What does atoi("123") return?

A. 0

B. 123.0

C. 123

D. "123" (string)

Ans: C

Explanation: atoi() converts the digit string "123" directly into the integer value 123.

Q31 ATOI()

What does atoi("abc") return?

A. -1

B. A runtime error

C. 0

D. "abc" unchanged

Ans: C

Explanation: With no valid leading digits present, atoi() returns 0 by definition.

Q32 ATOI()

What does atoi("45abc") return?

A. "45abc" unchanged

B. 45

C. A runtime error

D. 0

Ans: B

Explanation: atoi() converts the leading numeric portion and stops as soon as it meets a non-digit character.

Q33 ATOI()

What does atoi(" -78") return, given the leading space?

A. -78

B. 0

C. A runtime error

D. 78

Ans: A

Explanation: atoi() skips leading whitespace first, then correctly parses the negative sign and digits.

Q34 ATOI()

What type of argument does atoi() require?

A. int*

B. const char* (a string)

C. single char

D. double

Ans: B

Explanation: atoi() accepts a pointer to a null-terminated character string as its argument.

Q35 ATOI()

Which of the following is a common mistake when using atoi()?

A. Passing a string literal as the argument

B. Storing the result in an int variable

C. Passing a non-numeric string and assuming the returned 0 means valid input of "0"

D. Including stdlib.h before using it

Ans: C

Explanation: Since atoi() returns 0 both for genuinely invalid input and for the valid string "0", this ambiguity is a frequent source of bugs.

Q36 ATOI()

What is the output of: printf("%d", atoi("3.9"));

A. 0

B. 3

C. 3.9

D. 4

Ans: B

Explanation: atoi() stops conversion as soon as it reaches the decimal point, so only the digit "3" is converted.

Q37 ATOI()

Which function is specifically designed to convert a whole-number string into type int for arithmetic use?

A. toupper()

B. atoi()

C. atof()

D. isdigit()

Ans: B

Explanation: atoi() is the standard function for converting integer-valued numeric strings into type int.

Q38 ATOI()

What does atoi() return for the input string "0"?

A. Undefined behaviour

B. A runtime error

C. 0

D. NULL

Ans: C

Explanation: The string "0" is validly parsed, and atoi() correctly returns the integer 0.

Q39 ATOI()

Assertion (A): atoi("007") returns 7.

Reason (R): atoi() ignores insignificant leading zeros when computing the numeric value.

A. A is false, R is true

B. Both A and R are true, and R correctly explains A

C. A is true, R is false

D. Both A and R are false

Ans: B

Explanation: Leading zeros do not change the numeric value of the digits that follow, so atoi() correctly returns 7.

Q40 ATOI()

Which statement about atoi()'s handling of very large numeric strings is correct?

A. It raises a compile-time error

B. It always returns INT_MAX in such cases

C. The behaviour is undefined if the value exceeds the range representable by int

D. It automatically promotes the result to type long

Ans: C

Explanation: atoi() performs no overflow checking, so converting a value outside int's range results in undefined behaviour.

Q41 ATOI()

What is the key difference in the outputs of atoi("12") and atof("12")?

A. atoi() returns a string, while atof() returns an int

B. atof() returns a string, while atoi() returns an int

C. atoi() returns the int value 12, while atof() returns the double value 12.0

D. Both functions return an identical type and value

Ans: C

Explanation: Although both parse the same digits, atoi() and atof() differ in the return type of the converted result.

Q42 ATOI()

Fill in the blank: atoi() converts the initial portion of a string to an ___ value.

A. boolean

B. integer

C. character

D. floating-point

Ans: B

Explanation: atoi() is specifically designed for converting a numeric string into an integer.

Q43 ATOI()

What is the result of atoi("+56")?

A. -56

B. A runtime error

C. 56

D. 0

Ans: C

Explanation: atoi() correctly interprets the optional leading '+' sign, so the result is a positive 56.

Q44 ATOI()

Which statement best describes atoi()'s behaviour on encountering a non-digit character in the middle of the string?

A. It raises a runtime exception

B. It immediately returns 0 regardless of prior digits

C. The non-digit character is skipped and parsing continues

D. Conversion stops at that character and the value parsed so far is returned

Ans: D

Explanation: atoi() halts conversion the moment it encounters a character that is not part of a valid integer.

toupper() - Convert to Uppercase

Q45 TOUPPER()

What is the purpose of toupper()?

A. To check if a character is uppercase

B. To convert an uppercase letter to lowercase

C. To convert a lowercase letter to its uppercase equivalent

D. To convert a character to a digit

Ans: C

Explanation: toupper() converts lowercase alphabetic characters to their corresponding uppercase form.

Q46 TOUPPER()

Which header file declares toupper()?

A. string.h

B. stdlib.h

C. ctype.h

D. stdio.h

Ans: C

Explanation: toupper() is a character-handling function declared in ctype.h.

Q47 TOUPPER()

What is the return type of toupper()?

A. char

B. int

C. double

D. void

Ans: B

Explanation: toupper() returns an int representing the (possibly converted) character.

Q48 TOUPPER()

What does toupper('a') return?

A. 'a' unchanged

B. A runtime error

C. 'A'

D. 97 (unchanged ASCII)

Ans: C

Explanation: toupper() converts the lowercase letter 'a' to its uppercase equivalent 'A'.

Q49 TOUPPER()

What does toupper('A') return, given an already uppercase input?

A. 'A' (unchanged)

B. 'a'

C. A runtime error

D. 0

Ans: A

Explanation: If the character is already uppercase, toupper() simply returns it unchanged.

Q50 TOUPPER()

What does toupper('5') return for a digit character?

A. 0

B. 'E'

C. A runtime error

D. '5' (unchanged)

Ans: D

Explanation: toupper() only affects lowercase alphabetic characters; non-letter characters are returned unchanged.

Q51 TOUPPER()

What argument type does toupper() expect?

A. int (representing a character)

B. float

C. char* (a string)

D. A string literal

Ans: A

Explanation: toupper() takes a single int argument representing the character code to convert.

Q52 TOUPPER()

What is the output of: printf("%c", toupper('z'));

A. Y

B. A runtime error

C. Z

D. z

Ans: C

Explanation: toupper() converts the lowercase letter 'z' to its uppercase equivalent 'Z'.

Q53 TOUPPER()

What happens if toupper() is applied to a special character like '@'?

A. It causes a runtime error

B. It is returned unchanged since it is not a lowercase letter

C. It is converted to a random letter

D. It is somehow converted to an uppercase '@'

Ans: B

Explanation: toupper() only converts lowercase letters; all other characters pass through unaffected.

Q54 TOUPPER()

Which approach correctly converts an entire string to uppercase, character by character?

A. Applying atoi() to the string

B. Applying isalpha() to the string

C. A single direct call to toupper() on the whole string

D. A loop that applies toupper() to each character of the string

Ans: D

Explanation: Since toupper() only works on one character at a time, converting a whole string requires iterating over it.

Q55 TOUPPER()

What is the output of: printf("%c", toupper('m'));

A. m

B. N

C. n

D. M

Ans: D

Explanation: toupper() converts the lowercase letter 'm' to its uppercase form 'M'.

Q56 TOUPPER()

Assertion (A): toupper() changes the case of alphabetic characters only.

```
Reason (R): Non-alphabetic characters, such as digits and symbols, have no case to change.
```

A. A is false, R is true

B. Both A and R are true, and R correctly explains A

C. A is true, R is false

D. Both A and R are false

Ans: B

Explanation: Since digits and symbols lack an upper/lower case distinction, toupper() leaves them unchanged.

Q57 TOUPPER()

Which of the following statements about toupper() is correct?

A. It converts numeric digits into letters

B. It permanently modifies the original character variable in memory

C. It always adds a fixed offset to any character's ASCII value

D. It returns the character unchanged if it is not a lowercase letter

Ans: D

Explanation: toupper() only alters lowercase letters; every other kind of input is returned unchanged.

Q58 TOUPPER()

What is the result of toupper(toupper('b'))?

A. 'B'

B. A runtime error

C. 'C'

D. 'b' unchanged

Ans: A

Explanation: Applying toupper() a second time has no further effect since 'B' is already uppercase.

tolower() - Convert to Lowercase

Q59 TOLOWER()

What is the purpose of tolower()?

A. To check if a character is a letter

B. To convert a lowercase letter to uppercase

C. To convert digits into characters

D. To convert an uppercase letter to its lowercase equivalent

Ans: D

Explanation: tolower() converts uppercase alphabetic characters to their corresponding lowercase form.

Q60 TOLOWER()

Which header file declares tolower()?

A. ctype.h

B. stdio.h

C. stdlib.h

D. string.h

Ans: A

Explanation: tolower() is declared in ctype.h alongside the other character-handling functions.

Q61 TOLOWER()

What is the return type of tolower()?

A. float

B. char

C. int

D. void

Ans: C

Explanation: tolower() returns an int representing the (possibly converted) character.

Q62 TOLOWER()

What does tolower('X') return?

A. 'x'

B. 88 (unchanged ASCII)

C. 'X' unchanged

D. A runtime error

Ans: A

Explanation: tolower() converts the uppercase letter 'X' to its lowercase equivalent 'x'.

Q63 TOLOWER()

What does tolower('x') return when the input is already lowercase?

A. 0

B. 'X'

C. 'x' (unchanged)

D. A runtime error

Ans: C

Explanation: If the character is already lowercase, tolower() simply returns it unchanged.

Q64 TOLOWER()

What does tolower('9') return for a digit character?

A. '9' (unchanged)

B. 0

C. A runtime error

D. 'a'

Ans: A

Explanation: tolower() only affects uppercase letters; digit characters are returned unchanged.

Q65 TOLOWER()

What argument type does tolower() accept?

A. int (representing a character)

B. char* (a string)

C. double

D. A string literal

Ans: A

Explanation: tolower() takes a single int argument representing the character code to convert.

Q66 TOLOWER()

What is the output of: printf("%c", tolower('Q'));

A. A runtime error

B. Q

C. p

D. q

Ans: D

Explanation: tolower() converts the uppercase letter 'Q' to its lowercase equivalent 'q'.

Q67 TOLOWER()

What happens if tolower() is applied to a punctuation mark like '#'?

A. It causes undefined behaviour

B. It is converted into a lowercase letter

C. It is returned unchanged since it is not an uppercase letter

D. It causes a compilation error

Ans: C

Explanation: tolower() only converts uppercase letters; every other character remains unchanged.

Q68 TOLOWER()

Which construct is needed to convert an entire string to lowercase?

A. Applying atof() to the string

B. A loop that applies tolower() to each character of the string

C. Applying isalpha() to the string

D. A single call to tolower() passing the whole string

Ans: B

Explanation: tolower() works on only one character at a time, so a loop is required to process a full string.

Q69 TOLOWER()

What is the output of: printf("%c", tolower('D'));

A. e

B. d

C. D

D. c

Ans: B

Explanation: tolower() converts the uppercase letter 'D' to its lowercase equivalent 'd'.

Q70 TOLOWER()

Assertion (A): tolower() has no effect on digit characters.

```
Reason (R): Digit characters do not have an uppercase or lowercase form.
```

A. A is true, R is false

B. A is false, R is true

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: C

Explanation: Since digits lack case, tolower() leaves them completely unaffected.

Q71 TOLOWER()

Which statement correctly describes tolower()'s behaviour?

A. It always adds a fixed offset to every character's ASCII value

B. It returns the argument unchanged if it is not an uppercase letter

C. It permanently modifies the original variable passed to it

D. It works only on vowels

Ans: B

Explanation: tolower() converts only uppercase alphabetic characters; everything else is passed through unchanged.

Q72 TOLOWER()

What is the result of tolower(tolower('R'))?

A. A runtime error

B. 's'

C. 'r'

D. 'R' unchanged

Ans: C

Explanation: Applying tolower() a second time has no further effect since 'r' is already lowercase.

isalnum() - Alphanumeric Test

Q73 ISALNUM()

What is the purpose of isalnum()?

A. To check whether an entire string is numeric

B. To check whether a character is a letter only

C. To check whether a character is alphanumeric (a letter or a digit)

D. To convert a character into alphanumeric form

Ans: C

Explanation: isalnum() tests whether a character is either an alphabetic letter or a decimal digit.

Q74 ISALNUM()

Which header file declares isalnum()?

A. stdio.h

B. ctype.h

C. string.h

D. stdlib.h

Ans: B

Explanation: isalnum() is one of the character classification functions declared in ctype.h.

Q75 ISALNUM()

What is the return type of isalnum()?

A. char

B. int

C. bool

D. void

Ans: B

Explanation: isalnum() returns a nonzero integer to indicate true, or 0 to indicate false.

Q76 ISALNUM()

What does isalnum('7') return?

A. A runtime error

B. The character '7' itself

C. A nonzero value (true)

D. Zero (false)

Ans: C

Explanation: '7' is a digit character, so isalnum() classifies it as alphanumeric and returns nonzero.

Q77 ISALNUM()

What does isalnum('$') return?

A. The character '$' itself

B. A runtime error

C. Zero (false)

D. A nonzero value (true)

Ans: C

Explanation: '$' is neither a letter nor a digit, so isalnum() returns 0.

Q78 ISALNUM()

What does isalnum('K') return for an uppercase letter?

A. Zero (false)

B. A runtime error

C. The character 'K' itself

D. A nonzero value (true)

Ans: D

Explanation: 'K' is an alphabetic character, so isalnum() returns a nonzero (true) value.

Q79 ISALNUM()

What argument type does isalnum() take?

A. float

B. A string literal

C. int (representing a character)

D. char* (a string)

Ans: C

Explanation: isalnum() accepts a single int argument representing the character to be tested.

Q80 ISALNUM()

What is the output of: if(isalnum('_')) printf("Yes"); else printf("No");

A. No

B. Yes

C. A runtime error

D. Blank output

Ans: A

Explanation: The underscore character is not alphanumeric, so isalnum() evaluates to false.

Q81 ISALNUM()

A student assumes isalnum() checks only letters. What is incorrect about this assumption?

A. isalnum() checks both letters and digits, not just letters

B. isalnum() also treats whitespace as alphanumeric

C. isalnum() checks only digits, not letters

D. The assumption is completely correct

Ans: A

Explanation: isalnum() returns true for both alphabetic characters and numeric digits.

Q82 ISALNUM()

Which statement about isalnum() is correct?

A. It returns true only for digit characters

B. It returns true for spaces and punctuation marks too

C. It returns true for the ranges 'a'-'z', 'A'-'Z', and '0'-'9'

D. It converts non-alphanumeric characters into blanks

Ans: C

Explanation: isalnum() effectively combines the checks performed by isalpha() and isdigit().

Q83 ISALNUM()

What is the output of: printf("%d", isalnum('!') ? 1 : 0);

A. 33

B. 1

C. A runtime error

D. 0

Ans: D

Explanation: '!' is a punctuation symbol, not alphanumeric, so isalnum() evaluates to false (0).

Q84 ISALNUM()

Assertion (A): isalnum('A') and isalnum('9') both return nonzero values.

```
Reason (R): isalnum() considers both alphabetic and numeric characters as valid.
```

A. Both A and R are false

B. A is false, R is true

C. A is true, R is false

D. Both A and R are true, and R correctly explains A

Ans: D

Explanation: Since 'A' is a letter and '9' is a digit, both satisfy the isalnum() test.

Q85 ISALNUM()

Which of these characters would isalnum() classify as false?

A. '3'

B. ' ' (a blank space)

C. 'Z'

D. 'm'

Ans: B

Explanation: A blank space is neither a letter nor a digit, so isalnum() returns false for it.

Q86 ISALNUM()

What is a practical use of isalnum()?

A. Terminating input validation midway

B. Validating that a username contains only letters and digits

C. Converting a password to uppercase

D. Measuring the length of a string

Ans: B

Explanation: isalnum() is commonly used in input validation to ensure that only alphanumeric characters are present.

isalpha() - Alphabetic Test

Q87 ISALPHA()

What is the purpose of isalpha()?

A. To check whether a character is a digit

B. To check whether an entire string contains only letters

C. To check whether a character is an alphabetic letter

D. To convert a character into a letter

Ans: C

Explanation: isalpha() tests whether a given character falls in the alphabetic range (a-z or A-Z).

Q88 ISALPHA()

Which header file declares isalpha()?

A. stdio.h

B. stdlib.h

C. math.h

D. ctype.h

Ans: D

Explanation: isalpha() is part of the character classification group declared in ctype.h.

Q89 ISALPHA()

What is the return type of isalpha()?

A. char

B. int

C. bool

D. void

Ans: B

Explanation: isalpha() returns a nonzero value if the test is true, or 0 if it is false.

Q90 ISALPHA()

What does isalpha('G') return?

A. A nonzero value (true)

B. A runtime error

C. The character 'G' itself

D. Zero (false)

Ans: A

Explanation: 'G' is an alphabetic character, so isalpha() returns a nonzero value.

Q91 ISALPHA()

What does isalpha('4') return for a digit?

A. A nonzero value (true)

B. The character '4' itself

C. A runtime error

D. Zero (false)

Ans: D

Explanation: Digits are not classified as alphabetic, so isalpha() returns 0 for '4'.

Q92 ISALPHA()

What does isalpha(' ') return for a blank space character?

A. Undefined behaviour

B. A nonzero value (true)

C. Zero (false)

D. A runtime error

Ans: C

Explanation: A blank space is not a letter, so isalpha() correctly returns false.

Q93 ISALPHA()

What argument type does isalpha() accept?

A. double

B. int (representing a character)

C. A string literal

D. char* (a string)

Ans: B

Explanation: isalpha() takes a single character, represented as an int, as its argument.

Q94 ISALPHA()

What is the output of: printf("%d", isalpha('z') != 0);

A. 122 always

B. 0

C. The character 'z' itself

D. 1

Ans: D

Explanation: 'z' is alphabetic, so isalpha() returns a nonzero value, and the comparison against 0 evaluates to 1 (true).

Q95 ISALPHA()

Which incorrect belief do students often have about isalpha()?

A. Believing that isalpha() also returns true for digit characters

B. Believing isalpha() returns an int

C. Believing isalpha() is declared in ctype.h

D. Believing isalpha() checks letters only

Ans: A

Explanation: isalpha() strictly excludes digits; only alphabetic characters satisfy this test.

Q96 ISALPHA()

Which statement about isalpha() is correct?

A. It distinguishes alphabetic letters from all other character types, including digits

B. It checks specifically for whitespace characters

C. It converts non-letters into letters

D. It treats digits as letters

Ans: A

Explanation: isalpha() is used purely to detect whether a character is alphabetic.

Q97 ISALPHA()

What is the output of: if(isalpha('&')) printf("Letter"); else printf("Not a letter");

A. Blank output

B. A runtime error

C. Letter

D. Not a letter

Ans: D

Explanation: '&' is a symbol, not a letter, so isalpha() evaluates to false.

Q98 ISALPHA()

Assertion (A): isalpha('a') and isalpha('Z') both return true.

Reason (R): isalpha() checks for both lowercase and uppercase alphabetic characters.

A. Both A and R are true, and R correctly explains A

B. A is true, R is false

C. A is false, R is true

D. Both A and R are false

Ans: A

Explanation: isalpha() covers the entire alphabetic range, including both lowercase and uppercase letters.

Q99 ISALPHA()

What is a typical use case for isalpha()?

A. Converting a numeric string to a number

B. Terminating a program on invalid input

C. Validating that a name field contains only alphabetic characters

D. Checking whether a number is even

Ans: C

Explanation: isalpha() is commonly used in input validation to ensure that a field contains only letters.

isdigit() - Decimal Digit Test

Q100 ISDIGIT()

What is the purpose of isdigit()?

A. To convert a character into a digit

B. To check whether an entire string is numeric

C. To check whether a character is alphabetic

D. To check whether a character is a decimal digit (0-9)

Ans: D

Explanation: isdigit() tests whether a character falls within the range '0' to '9'.

Q101 ISDIGIT()

Which header file declares isdigit()?

A. stdio.h

B. stdlib.h

C. ctype.h

D. string.h

Ans: C

Explanation: isdigit() belongs to the character classification group declared in ctype.h.

Q102 ISDIGIT()

What is the return type of isdigit()?

A. bool

B. char

C. void

D. int

Ans: D

Explanation: isdigit() returns a nonzero value for true, and 0 for false.

Q103 ISDIGIT()

What does isdigit('5') return?

A. A runtime error

B. Zero (false)

C. A nonzero value (true)

D. The character '5' itself

Ans: C

Explanation: '5' is a decimal digit, so isdigit() returns a nonzero value.

Q104 ISDIGIT()

What does isdigit('B') return for a letter?

A. A runtime error

B. A nonzero value (true)

C. The character 'B' itself

D. Zero (false)

Ans: D

Explanation: Letters are not digits, so isdigit() returns 0 for 'B'.

Q105 ISDIGIT()

What does isdigit('.') return for a decimal point character?

A. Undefined behaviour

B. Zero (false)

C. A runtime error

D. A nonzero value (true)

Ans: B

Explanation: A period is a punctuation mark, not a digit, so isdigit() returns false for it.

Q106 ISDIGIT()

What argument type does isdigit() accept?

A. float

B. char* (a string)

C. int (representing a character)

D. A string literal

Ans: C

Explanation: isdigit() takes a single character, passed as an int, as its argument.

Q107 ISDIGIT()

What is the output of: printf("%d", isdigit('0') != 0);

A. 48 always

B. 0

C. 1

D. The character '0' itself

Ans: C

Explanation: '0' is a valid digit character, so isdigit() returns a nonzero result, giving 1 after the comparison.

Q108 ISDIGIT()

What incorrect assumption do students often make about isdigit()?

A. Assuming isdigit() returns an int

B. Assuming isdigit() checks the numeric value of an int variable rather than a character code

C. Assuming isdigit() is declared in ctype.h

D. Assuming isdigit() takes one argument

Ans: B

Explanation: isdigit() checks whether a character symbol represents a digit; it is not meant to test whether an integer's value lies between 0 and 9.

Q109 ISDIGIT()

Which statement about isdigit() is correct?

A. It returns true for any positive numeric value

B. It returns true for both letters and digits

C. It returns true only for the characters '0' through '9'

D. It converts a digit character into its numeric value

Ans: C

Explanation: isdigit() strictly checks only for decimal digit characters.

Q110 ISDIGIT()

What is the output of: if(isdigit('9')) printf("Digit"); else printf("Not digit");

A. Not digit

B. Digit

C. Blank output

D. A runtime error

Ans: B

Explanation: '9' is a valid decimal digit, so isdigit() evaluates to true.

Q111 ISDIGIT()

Assertion (A): isdigit() and isalnum() both return true for the character '3'.

```
Reason (R): isalnum() includes the same digit test performed by isdigit().
```

A. Both A and R are true, and R correctly explains A

B. A is false, R is true

C. A is true, R is false

D. Both A and R are false

Ans: A

Explanation: Since '3' is a digit, both isdigit() and isalnum() classify it as true.

Q112 ISDIGIT()

What is a common practical use of isdigit()?

A. Terminating the program on invalid data

B. Measuring the length of a string

C. Validating that a user has entered only numeric characters in an input field

D. Converting text to uppercase

Ans: C

Explanation: isdigit() is frequently used character-by-character to confirm that input consists purely of digits.

exit() - Program Termination

Q113 EXIT()

What is the purpose of exit()?

A. To exit only the current loop

B. To terminate program execution immediately

C. To restart the program from the beginning

D. To pause program execution temporarily

Ans: B

Explanation: exit() terminates the entire program, not just the current function or loop.

Q114 EXIT()

Which header file declares exit()?

A. stdlib.h

B. stdio.h

C. process.h

D. ctype.h

Ans: A

Explanation: In standard C, exit() is declared in stdlib.h.

Q115 EXIT()

What is the argument type of exit()?

A. char*

B. int

C. void

D. double

Ans: B

Explanation: exit() takes a single integer status code as its argument.

Q116 EXIT()

What does exit(0) typically indicate?

A. An infinite loop was detected

B. An error occurred during execution

C. The program crashed unexpectedly

D. Successful program termination

Ans: D

Explanation: A status of 0, equivalent to EXIT_SUCCESS, conventionally signals successful completion.

Q117 EXIT()

What does exit(1) (a nonzero status) typically indicate?

A. A harmless warning message

B. Abnormal or unsuccessful program termination

C. Successful termination

D. The program is merely paused

Ans: B

Explanation: A nonzero exit status conventionally indicates that some error or abnormal condition occurred.

Q118 EXIT()

What does exit() return to the code that called it?

A. Nothing; it never returns to the caller

B. A pointer to a status message

C. The status code, as an int

D. A string describing the result

Ans: A

Explanation: exit() halts execution entirely, so control never returns to the calling code; the status is passed to the operating system instead.

Q119 EXIT()

Which two standard macros are commonly passed as arguments to exit()?

A. SUCCESS and ERROR

B. EXIT_SUCCESS and EXIT_FAILURE

C. TRUE and FALSE

D. NULL and EOF

Ans: B

Explanation: stdlib.h defines the macros EXIT_SUCCESS (0) and EXIT_FAILURE (typically 1) for use with exit().

Q120 EXIT()

What happens to open file streams when exit() is called?

A. They cause an immediate crash

B. They remain open indefinitely

C. They are deleted from the disk

D. They are automatically flushed and closed

Ans: D

Explanation: As part of its cleanup, exit() flushes output buffers and closes open streams before the program terminates.

Q121 EXIT()

What is the output/flow of: int main(){ printf("A"); exit(0); printf("B"); return 0; }

A. Only "A" is printed; "B" is never executed

B. A compilation error occurs

C. Only "B" is printed

D. Both "A" and "B" are printed

Ans: A

Explanation: exit(0) terminates the program immediately after "A" is printed, making the following statement unreachable.

Q122 EXIT()

What is wrong with calling exit() inside a function that is expected to return a value for further processing?

A. The program terminates immediately, so no value is ever returned to the caller

B. It causes a compile-time error

C. It automatically returns 0 to the caller function

D. It only exits the current function, not the program

Ans: A

Explanation: exit() ends the entire program at once, bypassing the normal function-return mechanism.

Q123 EXIT()

Which of the following statements about exit() is true?

A. exit() only works inside loops

B. exit() always requires two arguments

C. exit() can be called from any function, not just main()

D. exit() can only be called from within main()

Ans: C

Explanation: exit() is a general-purpose function and can terminate the program from anywhere in the code.

Q124 EXIT()

What is the key difference between calling exit() and using a plain return statement inside main()?

A. exit() can be called from any function and always terminates the whole program, while return only ends main() when used there

B. return performs cleanup, while exit() does not

C. exit() cannot accept any argument

D. They behave identically in every situation

Ans: A

Explanation: A return inside a non-main function merely exits that function, whereas exit() always terminates the entire program.

Q125 EXIT()

What is the output/flow of: int main(){ int x=5; if(x==5) exit(0); printf("%d",x); return 0; }

A. It prints 5

B. A compilation error occurs

C. Nothing is printed; the program terminates before the printf statement executes

D. It prints 0

Ans: C

Explanation: Since x==5 is true, exit(0) executes and the printf statement is never reached.

Q126 EXIT()

Which of the following is a valid practical use of exit()?

A. Testing whether a character is a digit

B. Terminating a program early when a critical file fails to open

C. Skipping a single iteration of a loop

D. Converting a string to a number

Ans: B

Explanation: exit() is commonly used to safely end a program when a critical error, such as a failed file operation, occurs.

Q127 EXIT()

Assertion (A): exit() invokes any functions registered with atexit() before the program terminates.

```
Reason (R): exit() is designed to perform proper cleanup before ending the program.
```

A. A is true, R is false

B. Both A and R are false

C. Both A and R are true, and R correctly explains A

D. A is false, R is true

Ans: C

Explanation: exit() calls atexit()-registered functions as part of its standard cleanup procedure before termination.

Q128 EXIT()

Fill in the blank: exit() passes its integer argument as the program's ___ to the operating system.

A. return address

B. memory address

C. process ID

D. exit (termination) status

Ans: D

Explanation: The integer argument supplied to exit() becomes the exit status that is reported to the operating system.

Comparison of Listed Functions

Q129 COMPARISON

What is the key difference between atof() and atoi()?

A. atof() returns an int, while atoi() returns a double

B. Both functions return exactly the same type

C. atoi() handles decimals, while atof() does not

D. atof() returns a double, while atoi() returns an int

Ans: D

Explanation: atof() is designed for floating-point conversion, whereas atoi() is designed for integer conversion.

Q130 COMPARISON

Which function correctly converts the string "3.14" without losing its fractional part?

A. atof()

B. isdigit()

C. toupper()

D. atoi()

Ans: A

Explanation: atoi() would truncate the value at the decimal point, while atof() preserves the fractional part.

Q131 COMPARISON

What is the main difference between toupper() and tolower()?

A. Both functions work only on digit characters

B. Both functions perform the exact same conversion

C. toupper() only tests case, while tolower() converts it

D. toupper() converts lowercase letters to uppercase, while tolower() converts uppercase letters to lowercase

Ans: D

Explanation: toupper() and tolower() perform exactly opposite case conversions.

Q132 COMPARISON

If a character is already in its target case, how do toupper() and tolower() behave?

A. Both raise a runtime error

B. Both convert the character into a digit

C. Their behaviour becomes unpredictable

D. Both simply return the character unchanged

Ans: D

Explanation: Neither function alters a character if the requested conversion does not apply to it.

Q133 COMPARISON

What is the key difference between isalnum() and isalpha()?

A. Both functions check only for digits

B. isalnum() checks for letters and digits, while isalpha() checks for letters only

C. They are functionally identical to each other

D. isalpha() checks for letters and digits, while isalnum() checks letters only

Ans: B

Explanation: isalnum() performs a broader check that includes both letters and digits, unlike isalpha().

Q134 COMPARISON

What is the key difference between isalpha() and isdigit()?

A. isalpha() tests for digits, while isdigit() tests for letters

B. isalpha() tests for alphabetic characters, while isdigit() tests for decimal digit characters

C. Both functions test exactly the same condition

D. isdigit() also accepts alphabetic characters

Ans: B

Explanation: These two functions test mutually exclusive categories of characters.

Q135 COMPARISON

Which relationship correctly describes isalnum(), isalpha(), and isdigit()?

A. isalnum() is completely unrelated to isalpha() and isdigit()

B. isalpha() internally includes the check performed by isdigit()

C. isdigit() internally includes the check performed by isalpha()

D. isalnum() returns true whenever either isalpha() or isdigit() would return true for that character

Ans: D

Explanation: isalnum() effectively combines the results of the isalpha() and isdigit() tests.

Q136 COMPARISON

Which grouping correctly distinguishes conversion functions from testing functions among those in this syllabus?

A. atof(), atoi(), toupper(), and tolower() perform conversions; isalnum(), isalpha(), and isdigit() perform tests

B. exit() is classified as a conversion function

C. All eight listed functions perform conversions

D. All eight listed functions perform tests only

Ans: A

Explanation: Conversion functions transform a value, while testing functions merely classify a character as true or false.

Q137 COMPARISON

How does exit() differ fundamentally from the other seven functions in this syllabus?

A. exit() has no defined practical use

B. exit() terminates the program rather than converting or testing a value

C. exit() converts a string into a number

D. exit() tests whether a character is valid

Ans: B

Explanation: Unlike the conversion and testing functions, exit() controls the flow and termination of the program.

Q138 COMPARISON

Which two functions both operate on a single character and return an integer true/false result?

A. atoi() and exit()

B. isalpha() and isdigit()

C. atof() and atoi()

D. toupper() and tolower()

Ans: B

Explanation: isalpha() and isdigit() are both character-testing functions returning a Boolean-style integer.

Q139 COMPARISON

Which two functions both operate on a single character and return a converted character (as an int)?

A. toupper() and tolower()

B. isalpha() and isalnum()

C. atoi() and atof()

D. isdigit() and exit()

Ans: A

Explanation: toupper() and tolower() are both character-conversion functions that return the modified character as an int.

Q140 COMPARISON

Scenario: A program must validate that a PIN consists only of numeric characters and then convert it into a number. Which two functions are most appropriate together?

A. isalpha() and atof()

B. isdigit() for validation, followed by atoi() for conversion

C. exit() and isalnum()

D. toupper() and tolower()

Ans: B

Explanation: isdigit() checks that each character is numeric, after which atoi() converts the validated string into an integer.

Q141 COMPARISON

Which statement correctly compares atoi() and isdigit()?

A. Both functions perform exactly the same operation

B. atoi() only tests characters without converting them

C. atoi() converts a numeric string to an integer, while isdigit() only checks whether a single character is a digit

D. isdigit() converts characters into numbers

Ans: C

Explanation: atoi() is a conversion function, while isdigit() is a testing function; the two serve different purposes.

Q142 COMPARISON

Which function(s) would be used together to build a case-insensitive comparison of two characters?

A. tolower() (or toupper()) applied to both characters before comparing them

B. exit() and isalnum()

C. atoi() and atof()

D. isalpha() and isdigit()

Ans: A

Explanation: Converting both characters to the same case before comparison ensures the comparison is case-insensitive.

Code and Output-Based Questions

Q143 CODE/OUTPUT

What is the output?

```
char c = 'p';
printf("%c", toupper(c));
```

A. p

B. Q

C. P

D. A runtime error

Ans: C

Explanation: toupper() converts the lowercase 'p' into its uppercase equivalent 'P'.

Q144 CODE/OUTPUT

What is the output?

```
printf("%d", atoi("25") + atoi("15"));
```

A. 25

B. 2515

C. 40

D. A runtime error

Ans: C

Explanation: Both strings are converted to integers 25 and 15, and their sum is computed as 40.

Q145 CODE/OUTPUT

What is the output?

```
printf("%.2f", atof("10.5") + atof("2.25"));
```

A. 12.75

B. 10.52.25

C. 12.50

D. A runtime error

Ans: A

Explanation: The two numeric strings are converted to doubles and added together, giving 12.75.

Q146 CODE/OUTPUT

What is the output?

```
if(isdigit('7')) printf("Digit"); else printf("NotDigit");
```

A. NotDigit

B. Digit

C. 7

D. A runtime error

Ans: B

Explanation: '7' is a valid digit character, so isdigit() returns true.

Q147 CODE/OUTPUT

What is the output?

```
char ch='M';
printf("%c", tolower(ch));
```

A. M

B. m

C. A runtime error

D. n

Ans: B

Explanation: tolower() converts the uppercase letter 'M' into its lowercase form 'm'.

Q148 CODE/OUTPUT

What is the output?

```
if(isalpha('9')) printf("Alpha"); else printf("NotAlpha");
```

A. NotAlpha

B. Alpha

C. A runtime error

D. 9

Ans: A

Explanation: '9' is a digit rather than a letter, so isalpha() returns false.

Q149 CODE/OUTPUT

What is the output?

```
printf("%d", isalnum('#') != 0);
```

A. 1

B. 0

C. '#' itself

D. 35

Ans: B

Explanation: '#' is neither a letter nor a digit, so isalnum() returns false, giving 0 after the comparison.

Q150 CODE/OUTPUT

What is the output?

```
int main(){ printf("Start"); exit(0); printf("End"); }
```

A. End

B. StartEnd

C. Start

D. A compilation error

Ans: C

Explanation: exit(0) halts the program right after "Start" is printed, so "End" is never printed.

Q151 CODE/OUTPUT

What is the output?

```
printf("%d", atoi("12") == 12);
```

A. A runtime error

B. 12

C. 0

D. 1

Ans: D

Explanation: atoi("12") evaluates to 12, and the comparison 12==12 is true, printing as 1.

Q152 CODE/OUTPUT

What is the output?

```
char c = 'Z';
if(isalpha(c) && !isdigit(c)) printf("Letter"); else printf("Other");
```

A. Other

B. Z

C. A runtime error

D. Letter

Ans: D

Explanation: 'Z' is alphabetic and is not a digit, so the combined condition evaluates to true.

Q153 CODE/OUTPUT

What is the output?

```
printf("%.1f", atof("100"));
```

A. "100" unchanged

B. 100.0

C. 100

D. A runtime error

Ans: B

Explanation: atof() converts "100" into the double value 100.0, printed with one decimal place.

Q154 CODE/OUTPUT

What is the output?

```
int x = atoi("07");
printf("%d", x);
```

A. 07

B. 0

C. A runtime error

D. 7

Ans: D

Explanation: atoi() interprets "07" in decimal, ignoring the insignificant leading zero, and returns 7.

Q155 CODE/OUTPUT

What is the output?

```
char c='k';
printf("%c%c", toupper(c), tolower(toupper(c)));
```

A. Kk

B. kK

C. kk

D. KK

Ans: A

Explanation: toupper('k') yields 'K', and tolower('K') then yields back 'k', producing "Kk".

Q156 CODE/OUTPUT

Identify the error:

```
int n = atof("3.5");
printf("%d", n);
```

A. The code fails to compile entirely

B. The printf format specifier causes the program to crash

C. atof() cannot accept a string like "3.5"

D. Assigning the double result of atof() to an int truncates the value, printing 3 instead of 3.5

Ans: D

Explanation: Storing a double value in an int variable truncates its fractional part, so the output is 3, not 3.5.

Q157 CODE/OUTPUT

What is the output?

```
for(char c='a'; c<='c'; c++) printf("%c", toupper(c));
```

A. A runtime error

B. ABC

C. AbC

D. abc

Ans: B

Explanation: The loop applies toupper() to each of 'a', 'b', and 'c', converting them all to uppercase.

Q158 CODE/OUTPUT

What is the output/flow?

```
int main(){ if(isdigit('x')) exit(1); printf("Valid"); return 0; }
```

A. Nothing is printed

B. Invalid

C. A compilation error

D. Valid

Ans: D

Explanation: 'x' is not a digit, so isdigit() is false, exit(1) is skipped, and "Valid" is printed.

Q159 CODE/OUTPUT

What is the output?

```
printf("%d", atoi("  -5"));
```

A. -5

B. 5

C. A runtime error

D. 0

Ans: A

Explanation: atoi() skips the leading whitespace and correctly interprets the negative sign, returning -5.

Q160 CODE/OUTPUT

Trace the flow:

```
int main(){ char c='B'; if(isalpha(c)) { c = tolower(c); printf("%c", c);} else exit(1); return 0;}
```

A. 'b' is printed

B. A compilation error occurs

C. 'B' is printed

D. Nothing is printed; the program exits

Ans: A

Explanation: 'B' is alphabetic, so tolower() converts it to 'b', which is then printed.

Difficult and Tricky Questions

Q161 DIFFICULT/TRICKY

What happens when printf("%d", atoi("2147483648")); is executed, assuming a 32-bit int (where INT_MAX = 2147483647)?

A. It reliably prints 2147483648

B. The result is undefined/unpredictable due to integer overflow

C. It reliably prints -2147483648

D. It always prints 0

Ans: B

Explanation: The value exceeds INT_MAX for a 32-bit int, and since atoi() performs no overflow checking, the behaviour is undefined by the standard.

Q162 DIFFICULT/TRICKY

What does atof(" +.5e1") return?

A. A runtime error

B. 0.5

C. 0.0

D. 5.0

Ans: D

Explanation: The string represents +0.5 x 10^1 = 5.0; atof() correctly parses the leading whitespace, sign, missing integer digit, and exponent.

Q163 DIFFICULT/TRICKY

What is the output of: printf("%c", toupper('1'+1));

A. The characters '1'+1 printed literally

B. 1

C. 2

D. A runtime error

Ans: C

Explanation: '1'+1 evaluates to the integer value of the character '2' (ASCII 50); since 50 is not a lowercase letter, toupper() returns it unchanged, so '2' is printed.

Q164 DIFFICULT/TRICKY

What does isalpha(toupper('e')) evaluate to?

A. Zero (false)

B. A nonzero value (true)

C. A runtime error

D. 'E' itself

Ans: B

Explanation: toupper('e') produces 'E', and isalpha('E') is true because 'E' is an alphabetic character.

Q165 DIFFICULT/TRICKY

Assertion (A): atoi("3e2") returns 3, not 300.

Reason (R): atoi() does not recognise scientific/exponential notation and stops at the first non-digit character.

A. A is false, R is true

B. Both A and R are false

C. Both A and R are true, and R correctly explains A

D. A is true, R is false

Ans: C

Explanation: Unlike atof(), atoi() has no concept of exponents, so it stops converting as soon as it reaches 'e'.

Q166 DIFFICULT/TRICKY

What can be said about the output of: printf("%d", isdigit('0') + isdigit('9'));?

A. The result is always guaranteed to be exactly 2

B. The result is always 0

C. It prints "09"

D. The result is a nonzero value, since both characters are digits (the exact numeric total is implementation-defined but nonzero)

Ans: D

Explanation: Both '0' and '9' are digits, so isdigit() returns nonzero for each; the standard only guarantees a nonzero result, not a specific value like 1.

Q167 DIFFICULT/TRICKY

Why does atoi(" ") (a string of only spaces) return 0?

A. atoi() raises an error for whitespace-only input

B. atoi() rounds the spaces down to zero

C. Spaces are treated as the digit zero

D. After skipping the leading whitespace, no digit characters remain to convert

Ans: D

Explanation: Once the whitespace is skipped, atoi() finds no valid digits and therefore defaults to returning 0.

Q168 DIFFICULT/TRICKY

What is the output/flow?

```
int main(){ char c='5'; int v = c - '0'; if(isdigit(c)) printf("%d", v); else exit(1); }
```

A. 0

B. 5

C. The program exits without printing anything

D. 53

Ans: B

Explanation: isdigit('5') is true, and the expression c-'0' correctly computes the numeric value 5, which is then printed.

Q169 DIFFICULT/TRICKY

What is the output of: printf("%s", atoi("10") ? "Nonzero" : "Zero");

A. 10

B. A runtime error

C. Zero

D. Nonzero

Ans: D

Explanation: atoi("10") returns 10, which is treated as true (nonzero) in the conditional expression.

Q170 DIFFICULT/TRICKY

Why do toupper() and tolower() take and return int rather than char?

A. Because char cannot store any ASCII value at all

B. Because characters cannot be passed as function arguments in C

C. Purely to improve execution performance

D. To allow the special value EOF to be represented and to safely accommodate all unsigned char values

Ans: D

Explanation: Using int lets these functions correctly handle every possible unsigned char value as well as the special value EOF.

Q171 DIFFICULT/TRICKY

A program calls isalnum(c) where c is a negative value (other than EOF) on a system with signed char. What is the concern?

A. The function always safely returns false

B. There is no concern in this situation

C. Passing a negative value other than EOF to a ctype.h function results in undefined behaviour

D. The function always safely returns true

Ans: C

Explanation: ctype.h functions require the argument to be representable as unsigned char or equal to EOF; other negative values cause undefined behaviour.

Q172 DIFFICULT/TRICKY

What is the output/flow?

```
int main(){ if(!isalpha('7')) exit(0); printf("Reached"); }
```

A. "7" is printed

B. "Reached" is printed

C. A compilation error occurs

D. Nothing is printed; the program terminates

Ans: D

Explanation: isalpha('7') is false, so !isalpha('7') is true, causing exit(0) to run before the printf statement.

Q173 DIFFICULT/TRICKY

What does atof("1,234.5") actually convert?

A. The full intended value 1234.5

B. Only the leading "1", giving the result 1.0, since the comma is not valid in the numeric grammar

C. 0.0, because the format is considered fully invalid

D. It causes a runtime error

Ans: B

Explanation: Conversion halts at the first invalid character, which in this case is the comma immediately following the digit "1".

Q174 DIFFICULT/TRICKY

In the expression exit(isdigit('A')), what status value is effectively passed to exit()?

A. 1

B. 0, since isdigit('A') evaluates to false

C. An undefined value

D. The character 'A' itself

Ans: B

Explanation: isdigit('A') evaluates to 0 (false) because 'A' is not a digit, so exit() is effectively called with a status of 0.

Mixed OAVS TGT Practice

Q175 MIXED PRACTICE

Which function converts a string representing a floating-point number into a floating-point value?

A. atof()

B. isdigit()

C. tolower()

D. atoi()

Ans: A

Explanation: atof() converts the initial portion of a character string representing a floating-point number into a double value.

Q176 MIXED PRACTICE

Which function would you use to check that a piece of user input contains only letters?

A. isalpha()

B. isalnum()

C. isdigit()

D. atoi()

Ans: A

Explanation: isalpha() is specifically designed to test whether a character is alphabetic.

Q177 MIXED PRACTICE

Match the following functions to their correct category:

```
1. atoi()  2. isalpha()  3. toupper()  4. exit()
A. Character testing  B. Program termination  C. Integer conversion  D. Character conversion
```

A. 1-B, 2-D, 3-A, 4-C

B. 1-D, 2-B, 3-C, 4-A

C. 1-C, 2-A, 3-D, 4-B

D. 1-A, 2-C, 3-B, 4-D

Ans: C

Explanation: atoi() performs integer conversion, isalpha() performs character testing, toupper() performs character conversion, and exit() performs program termination.

Q178 MIXED PRACTICE

Which function should be used first to validate that a character is numeric before converting an entire numeric string with atoi()?

A. isdigit()

B. toupper()

C. exit()

D. isalpha()

Ans: A

Explanation: isdigit() is used to validate each character before the string is converted using atoi().

Q179 MIXED PRACTICE

What return type is shared by toupper(), tolower(), isalnum(), isalpha(), and isdigit()?

A. char

B. void

C. double

D. int

Ans: D

Explanation: All five of these ctype.h functions return their result as an int value.

Q180 MIXED PRACTICE

Statement: "atoi() and atof() both stop conversion at the first character that cannot be part of a valid number." Is this statement true or false?

A. True

B. True only for atoi()

C. True only for atof()

D. False

Ans: A

Explanation: Both functions parse only the initial valid numeric portion of the string and disregard anything beyond it.

Q181 MIXED PRACTICE

Scenario: A calculator program reads user input as a string and must support decimal numbers. Which function is most appropriate for conversion?

A. toupper()

B. isdigit()

C. atof()

D. atoi()

Ans: C

Explanation: atof() supports decimal (floating-point) values, whereas atoi() only handles whole numbers.

Q182 MIXED PRACTICE

Which of the following is NOT a valid use of exit()?

A. Using exit() with a nonzero status code

B. Using exit() with the macro EXIT_SUCCESS

C. Using exit() to terminate the program after a fatal error

D. Using exit() to skip just the current iteration of a for loop

Ans: D

Explanation: exit() terminates the entire program; skipping a single loop iteration should instead use 'continue', not exit().

Q183 MIXED PRACTICE

Fill in the blank: The function ___ tests whether a character is either a letter or a digit.

A. isalnum()

B. isalpha()

C. isdigit()

D. toupper()

Ans: A

Explanation: isalnum() is the combined alphanumeric test covering both letters and digits.

Q184 MIXED PRACTICE

What is wrong, if anything, with this code?

```
char c = 65;
c = toupper(c);
printf("%c",c);
```

A. There is no error; 65 is the ASCII code for 'A', which toupper() leaves unchanged since it is already uppercase

B. toupper() cannot accept an integer literal like 65

C. The printf format specifier is invalid here

D. The code fails to compile

Ans: A

Explanation: 65 corresponds to the uppercase letter 'A', so toupper() simply returns it unchanged.

Q185 MIXED PRACTICE

Which two functions from the syllabus are most useful together for building a robust integer-input validation routine?

A. toupper() and tolower()

B. isdigit() and atoi()

C. isalpha() and atof()

D. exit() and isalnum()

Ans: B

Explanation: isdigit() validates each character, while atoi() then performs the actual numeric conversion.

Q186 MIXED PRACTICE

Application-based: A program must display an error and terminate if a user enters a non-numeric age. Which two functions are relevant here?

A. atoi() and toupper()

B. isdigit() (or isalnum()) for validation and exit() for termination

C. tolower() and isalpha()

D. atof() and toupper()

Ans: B

Explanation: isdigit() checks the validity of the input characters, and exit() ends the program if the input is invalid.

Q187 MIXED PRACTICE

Which of the following statements is FALSE regarding the eight syllabus functions?

A. All eight functions are declared in the same single header file

B. atof(), atoi(), and exit() are declared in stdlib.h

C. exit() terminates the running program

D. toupper(), tolower(), isalnum(), isalpha(), and isdigit() are declared in ctype.h

Ans: A

Explanation: The eight functions are split between two headers, stdlib.h and ctype.h, not a single common header.

Q188 MIXED PRACTICE

A student writes: int n = atoi("12abc34"); What value is stored in n?

A. A compilation error occurs

B. 12

C. 0

D. 1234

Ans: B

Explanation: Conversion halts at the first non-digit character 'a', so only the leading "12" is converted.

Q189 MIXED PRACTICE

Which approach allows comparing two strings case-insensitively without permanently altering the original stored string?

A. Applying tolower() or toupper() temporarily to characters only during the comparison

B. Using isalpha() and isdigit() on the strings

C. Using atoi() and atof() on the strings

D. Using exit() and isalnum() on the strings

Ans: A

Explanation: Applying the case conversion only during comparison, rather than to the stored string, achieves case-insensitive matching without side effects.

Q190 MIXED PRACTICE

Which of the eight syllabus functions is unique in that it does not process character or string data at all?

A. isalpha()

B. atoi()

C. tolower()

D. exit()

Ans: D

Explanation: exit() deals purely with program termination and takes only an integer status code, unlike the other seven functions which all process characters or strings.